Community Detection

By Vincent Pestilli, Wesley Ni, and Marcus VanWerry

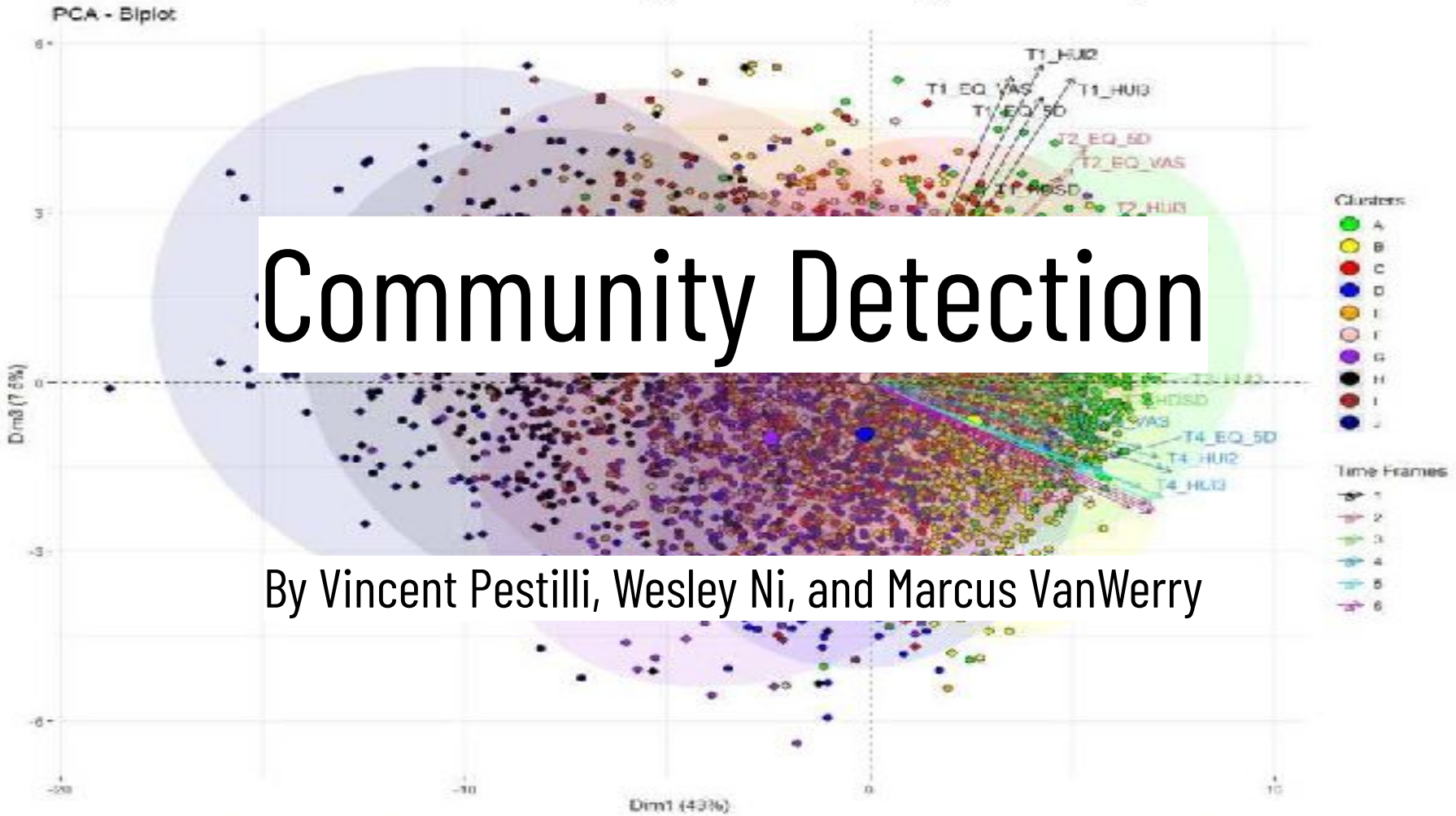


Figure 4: Existing Research in Patient Clustering Optimization

What is Community detection & What are its uses?

Applications of Community Detection

- **Social Network Analysis**
  - Identify friend groups, influence clusters, and information spread

    i. Social media like **Facebook, Twitter, etc.**

- **Biological Networks**
  - Detect functional modules in protein–protein interaction networks
  - Understand gene relationships and disease pathways

    ii. Used by the National Institute of health for **disease and genomics research**
    iii. 23andme for **genetic pattern analysis**

- **Recommendation Systems**
  - Group users with similar preferences
  - Improve content and product recommendations

    iv. Youtube, Amazon, Spotify, etc.

Code/Methods

Why Python?

- Fast to prototype and easy to scale
- Many libraries already exist
  - We don't need to reinvent the wheel
- **Strong support for matrices, arrays, and graph structures**

Key Libraries

- **NetworkX** → graph modeling, traversal, and community structures
- **NumPy** → fast matrix operations and linear algebra
- **scikit-learn** → K-means clustering implementation
- **Chrono** → Recording runtimes
- **Easygui** → Self-explanatory

Data Sets & Experimentation

- Randomly generated graphs for stress testing
  - 10, 100, 1000 nodes. 10%, 50%, 90% edge density
- Real world or custom datasets
  - Zachary's Karate Club
  - Football Conferences
  - Flower
- Recording and comparing runtimes and modularity scores
  - We ran our timing to 6 decimal points, even then some return 0.0 seconds elapsed

Approximate Time Complexities

Θ(n):

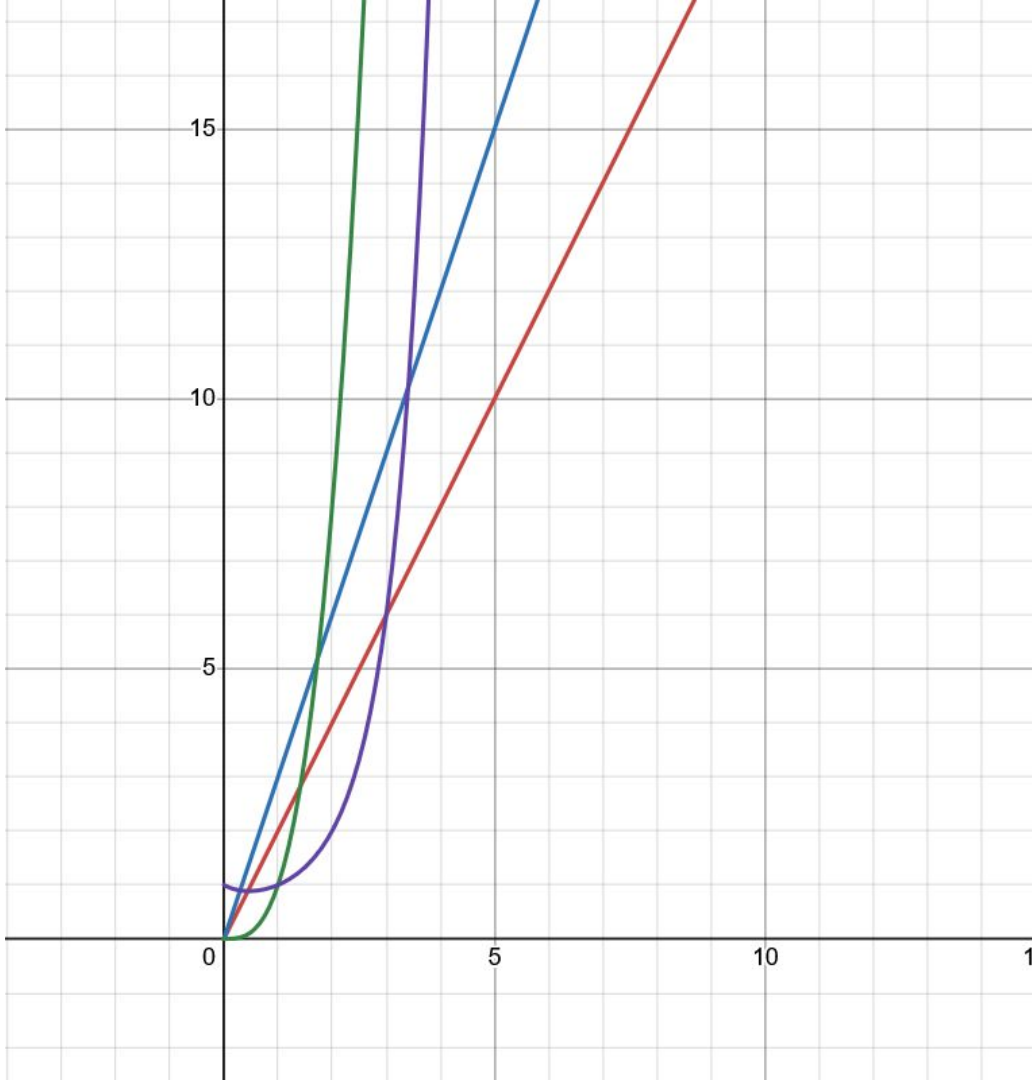
- Label Propagation (LPA)
- Emperor Penguin Optimization (EPO)

Θ(n^3):

- Spectral Clustering

Θ(n!):

- Brute Force



Modularity

Modularity measures how much more connected nodes are within their communities compared to what we'd expect in a random graph, scaled by the total number of edges. Higher values mean better community structure.

Simplified: "Good communities have more connections that expected by chance"

Modularity= (edges within communities – expected edges at random)/Total Edges

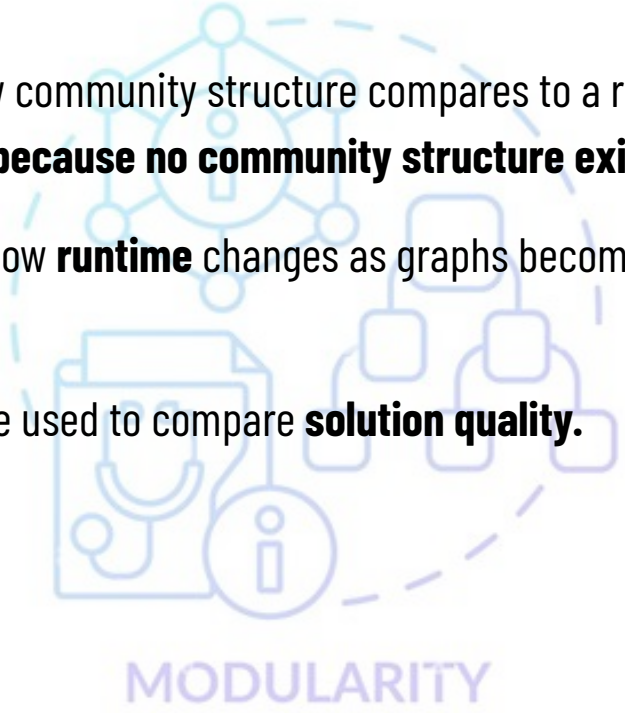
$$Q = \frac{1}{2m} \sum_{vw} \left[A_{vw} - \frac{k_v k_w}{2m} \right] \frac{s_v s_w + 1}{2}$$

Modularity - Experimental Limitation

Because modularity measures how community structure compares to a random baseline, **In fully random graphs, modularity is near zero because no community structure exists beyond chance.**

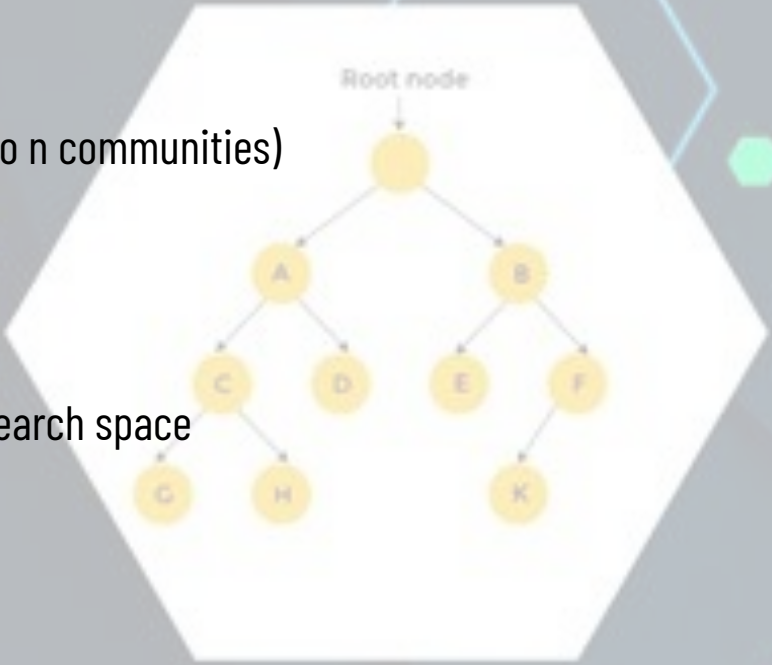
As such we are merely observing how **runtime** changes as graphs become **more dense** for the randomly generated graphs.

The additional, named datasets are used to compare **solution quality.**



Brute Force

- Generates **all possible node partitions** (from 1 community to n communities)
- Evaluates each partition using a **modularity score**
- Selects the partition with the **highest modularity**
- Exhaustive search guarantees the best solution within the search space
- **Time Complexity: ~ O(n!)**
- **Limitation:** explodes beyond small graphs (~n ≤ 10)



Empirical Analysis - Brute Force

Very fast on sparse graph (10–10 ⟶ 0.003s)

Runtime increases drastically with edge density

Modularity decreases as graph becomes denser

Collapses to 1 community at high density (10–90)

Algorithm	Dataset	Runtime(seconds)	Modularity	Communitie
Brute Force	10-10	0.003	0.22	
Brute Force	10-50	14.8	0.14	
Brute Force	10-90	16.58	0	

Label Propagation

- Gives each node a unique **label**
- Every **iteration**, each node gets assigned the most common label of its neighbors. Ties are broken randomly
- Continue until the graph is **stable** (no more label swaps)
- Very simple and fast, but inconsistent algorithm
- **Time Complexity: ~ O(n)**
- **Limitation:** Inconsistent, lower quality solutions due to randomness

Empirical Analysis - Label propagation

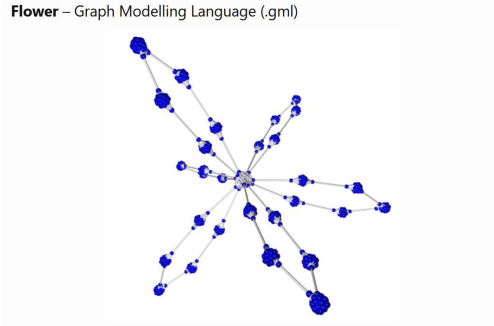
Near zero runtime across all datasets.
(Linear runtime)

As expected no meaningful structure was detected in random graphs. However runtime increases slightly with denser edges.

High modularity on Football and Flower data sets.

Very non deterministic with inconsistent community counts.

Flower – Graph Modelling Language (.gml)



Algorithm	Dataset	Runtime(seconds)	Modularity	Communities
Label Propagation(LPA)	10-10%	0	0	6
Label Propagation(LPA)	10-50%	0	0	1
Label Propagation(LPA)	10-90%	0	0	1
Label Propagation(LPA)	100-10%	0.006	0	1
Label Propagation(LPA)	100-50%	0.005	0	1
Label Propagation(LPA)	100-90%	0.007	0	1
Label Propagation(LPA)	1000-10%	0.113	0	1
Label Propagation(LPA)	1000-50%	0.556	0	1
Label Propagation(LPA)	1000-90%	0.68	0	1
Label Propagation(LPA)	Football	0.002	0.558	12
Label Propagation(LPA)	Football	0.001	0.59	9
Label Propagation(LPA)	Football	0.002	0.581	13
Label Propagation(LPA)	Flower	0.006	0.965	32
Label Propagation(LPA)	Flower	0.007	0.965	32
Label Propagation(LPA)	Flower	0.007	0.965	32
Label Propagation(LPA)	Zachary's Karate Club	0.001	0.399	4
Label Propagation(LPA)	Zachary's Karate Club	0	0.36	3
Label Propagation(LPA)	Zachary's Karate Club	0.001	0.133	3

Spectral Clustering

- Starts By represents graph using a **Laplacian matrix**
  - **A Laplacian matrix represents a graph's structure by combining its connections and node degrees, highlighting how each node is connected relative to the rest of the network**
  - Computes eigenvalues and eigenvectors to capture structure
  - Embeds nodes into lower dimensional space. **Connected nodes end up closer together**
  - Applies **clustering** (k-means) on embedded representation
  - Effective for **non convex and complex community shapes** (i.e., clusters that are not simple, round "blob-like" groups, but instead may be irregular, stretched, or indirectly connected shapes)
  - **Time Complexity**: **~ $O(n^3)$ (due to eigen decomposition)**
  - **Limitation**: expensive for large or dense graphs

Empirical Analysis - Spectral Clustering

Algorithm	Dataset	Runtime(seconds)	Modularity	Communities
Spectral Clustering	10-10	5.55	0	3
Spectral Clustering	10-50	0.024	0.0274	3
Spectral Clustering	10-50	0.008	-0.06	6
Spectral Clustering	10-90	0.023	-0.024	3
Spectral Clustering	100-10	0.048	0.218	3
Spectral Clustering	100-50	0.054	0.062	3
Spectral Clustering	100-50	0.014	0.04	6
Spectral Clustering	100-90	0.071	0.014	3
Spectral Clustering	1000-10	2.594	0.068	3
Spectral Clustering	1000-10	0.423	0.047	20
Spectral Clustering	1000-50	0.581	0.022	3
Spectral Clustering	1000-50	0.608	0.02	6
Spectral Clustering	1000-50	0.62	0.012	20
Spectral Clustering	1000-90	0.83	0.0065	3
Spectral Clustering	1000-90	0.822	0.002	20
Spectral Clustering	Flower	1.94	0.873	10
Spectral Clustering	Flower	0.317	0.963	30
Spectral Clustering	Flower	1.873	0.965	31
Spectral Clustering	Flower	0.304	0.892	50
Spectral Clustering	Football	1.589	0.601	12
Spectral Clustering	Football	0.014	0.577	6
Spectral Clustering	Zachary's Karate Club	0.008	0.273	10
Spectral Clustering	Zachary's Karate Club	0.008	-0.02	2
Spectral Clustering	Zachary's Karate Club	0.007	0.39	3

- Very fast in practice vs complexity class for datasets of the sizes we tested
- Often ran faster when k was higher
- Randomly very slow. Likely outliers due to external factors
- Often faster than EPO purely because of our awful implementation of EPO's iteration count

Emperor Penguin Optimization

Starts with creating “Penguins” or rather **random community assignments** (multiple potential solutions)

Iteratively improves solutions using:

- The **best current solution**
- **Local graph structure** (the immediate neighbors and connections surrounding a node)
- Controlled randomness

Through the use of a **temperature system**:

- High temperature ⟶ more exploration (random changes)
- Low temperature ⟶ more exploitation (convergence towards the best solutions)
- Temperature decreases over time

Gradually converges toward better community groupings over time
Strength: explores many possibilities and can escape poor solutions

Limitation: computationally heavy and not always consistent due to randomness

Empirical Analysis - Emperor Penguin Optimization

Algorithm	Dataset	Runtime(seconds)	Modularity	Communities
Emperor Penguin Optimizer (EPO)	10-10%	0.001	0.06	5
Emperor Penguin Optimizer (EPO)	10-50%	0.004	-0.045	8
Emperor Penguin Optimizer (EPO)	10-90%	0.006	-0.093	9
Emperor Penguin Optimizer (EPO)	100-10%	0.463	0.146	28
Emperor Penguin Optimizer (EPO)	100-50%	2.732	-0.007	66
Emperor Penguin Optimizer (EPO)	100-90%	4.988	-0.00899	65
Emperor Penguin Optimizer (EPO)	Football	0.3	0.578	8
Emperor Penguin Optimizer (EPO)	Football	0.3	0.589	8
Emperor Penguin Optimizer (EPO)	Football	0.31	0.59	10
Emperor Penguin Optimizer (EPO)	Zachary's Karate Club	0.02	0.15	16
Emperor Penguin Optimizer (EPO)	Zachary's Karate Club	0.02	0.016	17
Emperor Penguin Optimizer (EPO)	Zachary's Karate Club	0.018	0.073	19

- Showed significant slowdowns on larger and denser datasets due to our implementation of the iteration count
- We were not able to use the Flower dataset as runtime became too large.
- Further testing and better implementation would be necessary to draw any concrete conclusions on performance.

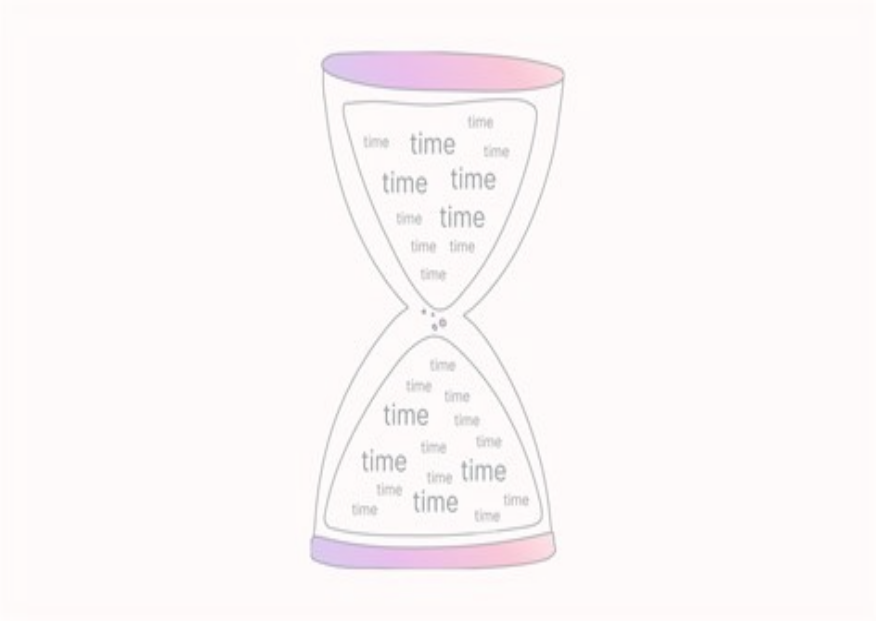
Algorithm Solution Quality Comparisons

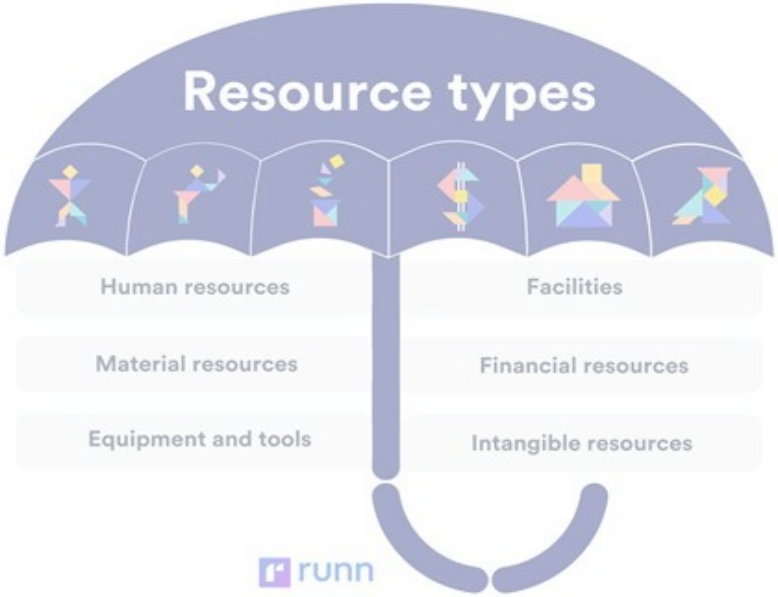
- LPA was the overall most accurate algorithm. More testing could show it to be less consistent due to randomness
- EPO was somewhat inconsistent, but due to its randomness, that will likely get ironed out with further experiments
- Spectral Clustering was a big underperformer. This is due to it being based on a k parameter that must be tuned. It doesn't have the randomness problem that the others do.

Modularity comparisons:	
Football:	**Modularity**
1. Spectral Clustering	0.589
2. EPO	0.586
3. LPA	0.576
Flower:	
1. LPA	0.965
2. Spectral Clustering	0.909
Zachary's Karate Club:	
1. LPA	0.297
2. Spectral Clustering	0.214
3. EPO	0.08

Future Improvements

- Improve implementations/make custom algorithm variations
- Experiment further with variant algorithms, different techniques, and parameters
- Improve our nature-inspired inspiration (EPO)





Works Cited

- Claude, ChatGPT*(Used for research, explanations, debugging, and programming assistance for LPA and EPO)*
- https://gist.github.com/pravj/9168fe52823c1702a07b
- https://www.sciencedirect.com/science/article/pii/S1110016822005361
- https://en.wikipedia.org/wiki/Label_propagation_algorithm
- https://www.geeksforgeeks.org/python/community-detection-in-social-networks-using-brute-force-method/
- https://networkdata.ics.uci.edu/netdata/html/zacharyKarate.html
  https://graphia.app/example-data.html
- https://networkx.org/documentation/stable/auto_examples/graph/plot_football.html

Questions?